Game sharing metadata and artwork

Read for either engine before editing sharing metadata or calling `webdev.config` POST `game/sharing/prepare`. Follow the recorded `gameEngine` and its preparation section below. Sharing preparation does not create a checkpoint, publish the game or authorize image generation.

Title consistency

For a remix, carry the new title chosen during setup into the game's visible title/menu, engine or browser title, loading screen, sharing metadata/captions and any title lettering in artwork.
Check these before the first complete checkpoint: importing source code or assets must not restore the source game's branding. Keep source credits, licenses and internal asset/save identifiers intact. A new name alone does not fulfill the requested gameplay changes. Routine edits to the resulting game retain its chosen name.

Game-specific loading screen

Every new game and remix replaces its inherited loading screen first, before gameplay work, for both engines and all effort levels. Replace inherited template/source-game backgrounds, logos and splash art; changing only the title over the old picture is insufficient. Reuse loading mechanics, not the source game's look.

Start with a solid-color background, the game's title and the loading progress bar; this first step needs no generated art. Before the first complete checkpoint, center the game's own favicon above the bar.
Use real progress, or an indeterminate bar for unknown totals, with readable status and Retry on failure. This simple design is sufficient. For a richer screen, create art for this game within recorded permissions or use this game's own OG/title art, never the template's cover.

Inspect the active loading artwork and references, including engine splash and in-game loading transitions, for inherited art.
Keep source credits, licenses and unused reference assets; disconnect old presentation

references rather than
deleting source files. Preserve the resulting game's own loader on routine edits; do not regenerate it each run.
Godot shell wiring and pre-PCK image availability follow delivery.

Project metadata

Sharing is project content, not a second candidate or acceptance API. Store it in `game-sharing.json`:

JSON


```
{"schemaVersion":2,"title":"Game title","description":"Play experience","defaul
```


The file and fields after `schemaVersion` are optional. Use regular PNGs under `assets/`, at most 8 MiB and 16M pixels each. Never use absolute, remote, signed, base64 or credential paths, including
Dashboard screenshot tickets. Save the metadata and artwork with the project; Three.js uses the managed asset flow below. UI Share text
is a personal draft, not a project revision. Edit only requested fields; text changes need no image generation.
Source drafts never change old checkpoints, and version 1 metadata stays readable.

OG cover and favicon

Honor supplied artwork, existing valid covers and the user's image-generation restrictions. Follow the selected engine's preparation below for production choices.

For an authorized generated cover, use actual game art or captures as references, normally 1200×630
with safe margins. Generate the exact game name as distinctive styled lettering within the artwork,
not a plain label pasted later. Check spelling and legibility, with at most two corrective image edits
per cover or rename. If still unsuitable, keep the previous valid cover or defer it and report; never block a valid checkpoint or claim success. A rename edits existing lettering; publishing,
Share edits or new guidance do not regenerate artwork. The loader may reuse the cover.
The checkpoint screenshot is separate from the OG cover and favicon.

Favicon

Every first complete game, including Godot Fast prototype and Three.js quick prototypes, needs a game-specific
favicon. Default engine or template icons do not satisfy this requirement. Reuse suitable existing artwork;
otherwise use a permitted recognizable game crop or adaptation, or a simple game-specific PNG for a prototype.
Keep prototype artwork bounded; this does not require AI generation, an OG cover or additional gameplay assets.
Preserve existing valid icons on routine changes and honor explicit no-artwork or no-AI restrictions.
Record the PNG as `assets.favicon` in `game-sharing.json`. When first adding sharing, also set game-specific
`title` and `description` instead of inheriting template text; preserve existing user wording on routine changes.
Check the icon at 32×32 and follow the engine's preparation below before the first complete checkpoint.
Verify the built favicon uses that image and the sharing title/description describe the actual game; report missing or invalid
artwork without blocking a valid playable checkpoint or claiming the favicon is complete.

Three.js preparation

Three.js has no Development Effort choice or concept stage. For every first complete Three.js game,
including quick prototypes, prepare the game-specific favicon as above. Prepare an OG cover unless the user
explicitly asks for a quick prototype or no artwork. The OG cover is a separately AI-generated promotional image; honor explicit supplied-cover
or no-AI restrictions and report limits rather than substituting a screenshot, local render or catalog
composite. Follow the shared artwork guidance above and preserve existing artwork on routine changes.
This sharing requirement does not authorize unsolicited generation of additional gameplay assets.

Before the first sharing build, call `webdev.config` POST `game/sharing/prepare` with `{}`.
It installs a portable sharing step in the existing `package.json` build script using the trusted publication origin. Run the current runtime asset sync after the artwork lands, following Three.js assets. Commit the metadata, manifest and installed
sharing/asset recovery scripts; managed PNG bytes stay in Webdev Storage.
Keep the project's Web build configuration: the starter uses `pnpm build` → `dist`.

Check the build output for the favicon link and its image file, plus OG/Twitter image tags and their image files
when an OG cover is included, before claiming sharing is ready. Report missing or invalid artwork without blocking a valid playable checkpoint or
claiming the artwork succeeded.

Repeat preparation after domain or build-configuration changes, save the updated files in a new
checkpoint and publish that checkpoint only when authorized. Use Git/checkpoints and Web deployment. Preparation or file edits alone do not update live cards.

Godot preparation

Title fallback: project.godot config/name, then Game. Register sharing images in the asset lock.
Routine edits retain existing sharing.

Standard effort requires a separately AI-generated OG cover in every production mode, including catalog/procedural,
under the generation gate. Honor supplied-cover/no-AI restrictions;
report limits, never substitute screenshots, local renders or catalog composites. Generate initially or on request
using the shared artwork guidance. Fast skips unrequested OG production.
The favicon follows the recorded game-art choice/prototype rules. Set project.godot config/icon to the game-specific
PNG in game-sharing.json, never the engine default. These production rules are Godot-only.

If game-sharing.json exists, webdev.config POST game/sharing/prepare with {} before release. Lost preparation state
requires prepare again before rebuilding. Build handles immutable sharing images/metadata and index tags;
invalid/missing sharing warns but does not block basic readiness. File edits alone are not publication.
A domain change does not block game publication:
afterwards prepare, rebuild, checkpoint and publish when authorized, reusing images.
Preparation does not update live cards.

Use Game delivery to save and publish the prepared game.